

MLWE’s impact on Web Metrics, mTLS TTLB, and AWS service endpoint connections

Mila Anastasova
Amazon Web Services
milaaws@amazon.com

Panos Kampanakis
Amazon Web Services
kpanos@amazon.com

Abstract—Migrating to quantum-resistant cryptographic algorithms, specifically the NIST-standardized Module Learning with Errors (MLWE) primitives, would inevitably result in data transmission overhead in secure transport protocols due to their larger key, ciphertext, and signature sizes. Would the connection setup cost noticeably affect application performance? This study evaluates MLWE’s performance impact on practical use cases that rely on TLS 1.3 via real-world experiments. We analyze three distinct scenarios by sharing empirical and experimental data of applications interfacing with cloud service TLS endpoints, Web user metrics, and mutual TLS connections. We argue that some cloud applications will not be significantly affected due to their unconstrained environment. We show that Web performance degradation will remain below 10% for common webpages, corresponding to time delays of under 100ms, which users are unlikely to perceive. For mutual TLS applications, our experiments show that MLWE noticeably affects Time-to-First-Byte, almost doubling the connection times compared to plain TLS. However, when evaluating Time-to-Last-Byte, a metric more closely tied to application performance, the overall impact drops to about 15% for ~150KB data transfers in fast or slow networks. This impact is much lower for large client-server round-trips. While these results are reassuring that MLWE could unnoticeably be introduced in common TLS use cases, they do not diminish the value of data trimming techniques proposed in the literature (e.g., session resumption, intermediate certificate authority suppression) to speed up connections.

I. INTRODUCTION

Advances in quantum computing pose a threat to the current asymmetric cryptographic algorithms widely used on the Internet. If a cryptographically relevant quantum computer is developed, Shor’s algorithm [1] could break public key primitives based on integer factorization and (elliptic curve) discrete logarithm problems. These include RSA, (Elliptic Curve) Diffie-Hellman ((EC)DH), and Digital Signature Algorithm ((EC)DSA).

In the search for novel algorithms robust to quantum adversaries, lattice-based cryptography has emerged as a promising candidate. In particular, Module Learning with Errors (MLWE) schemes are a subcategory of the Learning with Errors primitives which offer relatively shorter keys and signature sizes and competitive performance. In 2024, the United States National Institute of Standards and Technology (NIST) [2] released two MLWE standards: Module Lattice-based Key Encapsulation Mechanism (ML-KEM) [3]) as a quantum-resistant key encapsulation algorithm and Module Lattice-based Digital Signature Algorithm (ML-DSA) [4])

as a signing primitive. ML-KEM and ML-DSA offer three parameter sets each: ML-KEM-512, ML-KEM-768, and ML-KEM-1024 and ML-DSA-44, ML-DSA-65, and ML-DSA-87. These new algorithms have already been deployed in diverse applications by stakeholders on the Web or other platforms that run over TLS 1.3 or SSH.

In secure transport protocols, the computational cost of ML-KEM and ML-DSA remains similar to classical (EC)DH and RSA/ECDSA; however, they feature larger public key, ciphertext, and signature sizes during connection establishment, which raises performance concerns due to increased data transmission cost. In the context of key exchange, the ML-KEM public key and ciphertext add approximately 1.5-2.2KB of cryptographic material over classical ECDH key exchanges. ML-DSA inflates the size of private PKI X.509 certificates from 1-2KB with RSA to 5-6KB per certificate, depending on the ML-DSA parameter set. WebPKI sizes, however, can reach up to 10-13KB due to the two (or more) Signed Certificate Timestamps (SCT). Thus, the total quantum-resistant authentication data transferred during connection setup could add up to 15-20KB for ML-DSA-44 or ML-DSA-65 [5].

When analyzing the impact of PQ cryptography on applications running over TLS 1.3, it is not sufficient to focus only on the connection setup latency. Applications or users do not directly notice a handshake; their perceived performance depends on multiple other factors. For example, Google’s Core Web Vitals metrics [6] define comprehensive criteria for evaluating web user experience. These focus on load latency, time to page interactivity, and visual content stability. Key metrics include Time-to-First-Byte (TTFB), Time-to-Last-Byte (TTLB), First Contentful Paint, Largest Contentful Paint, Load Time, and Fully Loaded Time.

While not prevalent on the Web, some TLS applications employ mutual authentication with Mutual TLS (mTLS). Examples include Zero Trust Architectures (ZTA) [7], WebVPN applications, various machine-to-machine interactions, microservice architectures, and API communications. mTLS connections rely on bidirectional authentication. Thus, they will incur the PQ key exchange and authentication data in both directions.

Our work builds on previous studies and presents new insights into the real-world impact of quantum-resistant MLWE-based TLS applications that go beyond the handshake overhead.

Our main contributions are:

- 1/ We share data about the 50th and 90th percentiles of TLS connections observed in three widely used AWS services. The data argues that the client application making these connections would be impacted by the PQ MLWE algorithms. However, it is unlikely to be noticeable due to the unconstrained conditions.
- 2/ We qualitatively investigate popular webpages and extrapolate the impact of ML-KEM and ML-DSA on common Web metrics across various network scenarios. We recognize that although the transition to MLWE adds significant overhead to the handshake, connection parallelization and large web content sizes amortize the cost and make the migration less noticeable to the user.
- 3/ We measure MLWE’s impact on web metrics of experimental webpages in different environments. The prototype webpages we tested consist of cached and uncached resources of various sizes. We emphasize that although the handshakes can become significantly slower, the overall impact on the user metrics is less significant, especially for faster networks, smaller client-server round-trips, and larger web content.
- 4/ We measure MLWE’s impact on the handshake and TTLB of mTLS applications. To our knowledge, this is the first study of post-quantum mTLS. We report significant impact when transferring small amounts of data over slow connections (~32% at 1Mbps) and showcase that fast network speeds, larger RTTs, and larger application data transfers mitigate the impact to the application/user.
- 5/ We show that round-trips introduced by congestion control are amortized by slow network speeds.

The rest of this paper is organized as follows: Section II presents an overview of the current literature. Section III provides the background on (m)TLS and describes important performance metrics beyond the handshake time. Section IV describes our experimental setup, network configuration parameters, and experiments. Section V includes our performance results and analysis. Section VI concludes this work with our key findings.

II. RELATED WORK

There are numerous studies evaluating the impact of new quantum-resistant algorithms to secure tunnel protocols like TLS 1.3, SSH, QUIC, VPNs, and more. Many of them focus on the handshake impact of large keys and authentication data exchanged and the CPU overhead. Early experimental deployments by Google and Cloudflare [8], [9], [10] showed that new post-quantum KEMs affect the tails of TLS 1.3 handshake time distributions. Crockett et al. [11] presented a survey of the challenges in deploying post-quantum cryptography across various Internet protocols and applications, emphasizing the size constraints, protocol-specific message limits, and varying latency across different network scenarios. Another early experimental study [12] showed that large keys and signatures affect the TLS 1.3 handshake especially at the 90th percentile under higher network loss probabilities.

Other works focused solely on authentication. Bindel et al. [13] studied the challenges of transitioning public key infrastructures (PKIs) to post-quantum algorithms, including certificate size increases, validation complexity, and compatibility with existing systems. [14] showed that data-heavy or CPU-heavy post-quantum signatures slow down the TLS 1.3 handshake and could even introduce an extra Round-Trip Time (RTT) due to TCP congestion control. The authors also proposed using different algorithms throughout the certificate chain in order to control the size and speed of the handshake. This optimization was also proposed in [15]. Various other works [16], [17], [18], [19], [20] have introduced methods to slim down the certificate chain exchanged in the handshake as a means for speeding up post-quantum connections. [21], on the other hand, evaluated the combination of post-quantum hybrid key exchanges and post-quantum signatures in TLS 1.3 and SSH and showed that well-performing quantum-resistant KEMs and signatures do not significantly degrade handshake performance under good network conditions and large congestion windows. Subsequently, Cloudflare experimentally tested [22] various TLS 1.3 certificate chain sizes and showed that, for their Web traffic profiles, more than 9-10KB of authentication data will slow down the tails of the handshake time distributions by double-digit percentages. In a similar vein, [23] argued that ML-DSA signatures and public keys are unacceptably large for TLS connections and would impose unacceptable delays on Web users.

More recent studies [24], [25], [26], [27] have confirmed the results of previous works, at least in the lower TLS handshake time percentiles. [28] developed an OpenSSL-based framework for assessing quantum-resistant and hybrid cryptographic primitives from the Open Quantum Safe project across different security levels and network conditions. Their conclusions are similar to previous studies.

Other analyses focused on constrained use cases. [29], [30] evaluated post-quantum algorithms for TLS in constrained environments and showed that although promising, they would introduce performance degradation. Tasopoulos et al. [31] evaluated the impact of algorithms in NIST’s PQC effort on timing, memory usage, and network traffic of TLS connections of lower-end, resource-constrained embedded devices. Their results demonstrate that, as expected, some PQ algorithms will impose substantially higher computational, memory, and communicational overhead than traditional algorithms. Building on their performance analysis, Tasopoulos et al. extended their investigation to examine the energy consumption characteristics of quantum-resistant TLS 1.3 implementations on ARM Cortex-M4 platforms [32]. They showed that the power consumption of the new algorithms can be lower than traditional ones depending on their processing performance. Additionally, [33] examined several wireless Internet of Things (IoT) communication networks and assessed the performance of post-quantum TLS over Bluetooth Low Energy (BLE) and WiFi. It showed that the balanced computational efficiency and moderate communication/storage overhead of ML-KEM and ML-DSA make them suitable for some IoT applications,

which is not the case for other schemes like Classic McEliece and SLH-DSA.

Although some publications [22], [23] have raised concerns about the Web user experience regression introduced by ML-DSA, these studies have focused on handshake times and did not quantitatively show the degradation of Web experience metrics. [34], [27] proposed the TTLB as a more realistic application performance metric. (**Full-disclosure**, these studies had an overlapping co-author with this work.) Subsequently, Kampanakis et al. [35] investigated the TTLB as more directly related to application or user experience and showed that the TLS 1.3 handshake overhead shrinks with high bandwidth, or as applications transfer more data and client-server round-trip times increase. [36] challenged this premise by sharing real-world aggregated data, which shows that Web content is served mostly by lean connections transferring small amounts of data. This could mean that the large size of post-quantum TLS handshakes will have a noticeable impact on Web performance, but this data cannot be directly assumed to apply on a per-webpage basis. A webpage that opens many small connections for servicing Web analytics collection and just a few large connections for the Web content could still fit in the reported profile, which does not necessarily mean that the user will notice the difference when the Web content is rendered. Thus, more data on connection sizes aggregated per webpage on the Internet is necessary to draw meaningful conclusions.

Finally, we note that in spite of all the TLS studies, to the best of our knowledge, there have not been any comprehensive quantitative studies looking at mTLS and the impact of new quantum-resistant algorithms on mTLS handshakes or other performance metrics. In this work, we address this gap.

III. PRELIMINARIES

A. (m)TLS 1.3 Handshake

TLS 1.3 is the most widely-deployed secure tunnel protocol, serving as the core of Internet communications. It establishes a secure channel between client and server via negotiating cryptographic parameters and authenticating the communicating parties. Figure 1 shows a summary of the typical (m)TLS 1.3 handshake. It starts with the `ClientHello` message, where the client sends its cryptographic key exchange capabilities and preferences to the server. This message contains supported ciphersuites and protocol versions and includes a `KeyShare` - the client public key. The server responds with a sequence of messages beginning with the `ServerHello`, which chooses cryptographic parameters from the client's capabilities and includes the server's own `KeyShare` for key agreement. The server also sends a `Certificate` message containing its digital certificate chain for authentication and the `CertificateVerify` message, a signature of the entire transcript, that proves the server is the owner of the private key corresponding to its identity certificate.

In the case of mTLS, authentication takes place bilaterally. The server requests for client authentication with a `CertificateRequest` message, and the client sends its `Certificate` chain and `CertificateVerify` signature

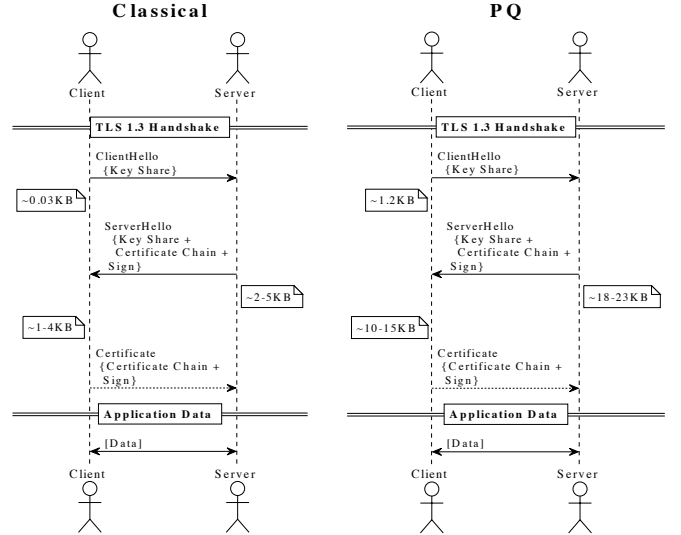


Fig. 1: Classical and PQ mTLS 1.3 handshakes

for the server to authenticate its identity. Client certificate authentication increases the data exchange size compared to plain TLS, as certificate chains are the largest exchanged messages in the handshake.

After the peers have established a shared secret from the key exchange and (unilaterally or mutually) authenticated, they can exchange encrypted data. The handshake concludes after `ChangeCipherSpec` and `Finished` messages.

Hybrid (PQ/T hybrid) handshakes, which combine Post-Quantum (PQ) and Traditional (T) key exchange, include ML-KEM public key and ciphertext along with classical ECDH `KeyShares`. Post-quantum authentication includes ML-DSA certificates and signatures in the handshake. ML-KEM and ML-DSA's processing performance is similar to traditional algorithms; however, they significantly increase the size of cryptographic data exchanged during the TLS handshake. Classical TLS handshakes transfer about 4-5KB of cryptographic data for a complete handshake. PQ TLS significantly increases this data to approximately 10-23KB, depending on the negotiated parameters and whether Private or WebPKI certificates are used. Figure 2 breaks down the cryptographic material sizes. An increase of about 15KB or more over classical plain TLS 1.3 handshakes raises questions about network performance, connection establishment latency, and the practical overhead of post-quantum deployments. mTLS, on the other hand, almost doubles the data size since certificate chains and signatures are transferred bidirectionally.

Most transport protocols include congestion control mechanisms to manage network congestion and to optimize throughput. TCP defines an initial congestion window (*icwnd*), which indicates the initial maximum number of TCP segments that can be transmitted before the sender pauses and awaits an acknowledgment from the receiver. The congestion window gets updated based on the congestion control mechanism throughout the life of the connection. At the time of this writing, the default *icwnd* is 10 Maximum Segment Size (MSS) packets as

	Classical Crypto			PQ Crypto	
	PK	Signature	Signed Timestamp (SCT)	PK	Signature
Root Certificate	RSA4096 (0.51)	RSA4096 (0.51)	–	MLDSA65 (2)	MLDSA65 (3.3)
Intermediate Certificate	RSA2048 (0.26)	RSA4096 (0.51)	–	MLDSA65 (2)	MLDSA65 (3.3)
Leaf Certificate	ECDSA P-256 (0.06)	RSA2048 (0.26)	2x ECDSA (0.13)	2x MLDSA44 (4.8)	MLDSA44 (1.3)
TLS Signature	–	ECDSA P-256 (0.06)	–	–	MLDSA44 (2.4)
TLS KeyShares	X25519 (0.03+0.03)			MLKEM768 (1.2+1.2)	
TLS Data	2.5			20.5	
mTLS Data	5			34	

Fig. 2: Approximate data size in KB of classical and MLWE (m)TLS 1.3 handshakes. The classical ones use X25519 key exchange, RSA4096 Root, 2048 Intermediate CA and ECDSA leaf WebPKI (with ECDSA SCTs) certs on the server. In mTLS, the client uses RSA4096 Root, 2048 Intermediate CA and ECDSA leaf Private PKI certs. The MLWE handshakes use X25519+MLKEM768 PQ/T key exchange, ML-DSA-65 Root and Intermediate CA, ML-DSA-44 leaf WebPKI (with ML-DSA-44 SCTs) certs on the server. In mTLS, the client uses ML-DSA-65 Root and Intermediate CA, ML-DSA-44 leaf WebPKI (with ML-DSA-44 SCTs) Private PKI.

defined in [37] (i.e., ~ 14.6 KB for common networks). From Figure 2, we can deduce that MLWE algorithms could exceed the default *icwnd* which can lead to additional round-trips and slow down connection establishment as also called out in [14], [21], [22], [34]. Similar congestion control and amplification protection techniques are employed in other secure transport protocols running over UDP. [38, §5.8.3] acknowledges that large certificates could cause congestion with DTLS 1.3, the UDP-based version of TLS, and offers alleviation options. QUIC [39] a newer UDP-based transport protocol used by HTTP3, defines similar congestion control mechanisms. All these techniques can introduce an additional round-trips for transmitting large ML-DSA certificate chains that exceed the defined congestion control limits.

While the transmission of the additional data in a post-quantum handshake may add a couple of milliseconds over high-speed networks (e.g., 100Mbps, 1Gbps), an additional round-trip could be significant. Thankfully, congestion control mechanisms have become more flexible [40], [41], [42, §Appendix C]. Senders could update their *icwnd*, depending on the network conditions, which could prevent the additional round-trip due to the ML-DSA certificate chain size. Content Distribution Networks (CDNs) already use such large *icwnds* [22] which speeds up their data transfers.

B. Performance Metrics

The research community has been focusing on handshake performance as an indicator of the performance impact the new MLWE algorithms will bring to applications running over secure tunnels. While the handshake time is important and is directly impacted by the negotiated cryptographic algorithms, it does not necessarily represent application performance. In

this work, we focus on metrics that are more indicative of application or user perceived performance. For example, applications usually open a secure connection, transfer data, and process it. The time it takes to receive and process this data time is often the relevant performance factor. The performance metric indicative of the data transfer is Time-To-Last-Byte (TTLB).

When it comes to Web user experience, it is measured by various metrics included in the Core Web Vitals [6] and CrUX Metrics [43]. These metrics capture aspects of the user experience such as loading speed, time to interactivity, and visual stability. In this work, we measure and analyze some of these metrics for various webpages to assess the impact the new algorithms will have on user experience. These include:

- **TTFB:** Time to receive the first resource of the content. The TTFB includes redirect times, DNS lookup, connection establishment, and the request-response and download time for the first fetched element.
- **First Contentful Paint (FCP):** Time to load any content, text, image, or non-white canvas. It is the time the user can start consuming Web content.
- **Largest Contentful Paint (LCP):** Time to load the largest image, text block, or video visible. It is the time the user perceives the main content is loaded.
- **Load or Document Complete (DC) time:** Time that the page `onLoad` event fires which is when the static Web content has loaded, excluding waiting for stylesheets, images, and subframes. It is an indicator of when the user perceives the full page to be rendered.
- **Fully Loaded time:** Time after the `onLoad` event has fired and network activity has stopped for 2 seconds. At this time the page and all associated resources have finished loading. It is an indicator of when the page is completely done.
- **Cumulative Layout Shift (CLS):** It tracks the visual stability of the page by measuring unexpected changes during loading the content. The higher the CLS, the less user-friendly the page is.

IV. EXPERIMENTS SETUP

In this work we focus on two different experimental approaches to evaluate the impact of PQ ML-KEM and ML-DSA on TLS 1.3 applications.

A. Web User Experience Experiments

Qualitative: First, we looked into the impact of post-quantum cryptography on Web browsing performance metrics. Naturally, it was not possible to enable both MLWE key exchange and authentication on popular websites, so we selected a diverse set of common websites and extrapolated the impact of the PQ algorithms by analyzing their connection and content profile. The websites included the landing page of search engines (`google.com`), social media platforms (`facebook.com`, `instagram.com`), productivity apps (`outlook.office.com`, `docs.google.com`), video streaming services (`youtube.com`), and news websites (`cnn.com`) as a diverse sample.

To collect Web user metrics, we used `webpagetest.org`, a Web performance measurement platform, run from Virginia, US. For each website, we established baseline performance numbers based on classical cryptographic primitives used today. Note that in order to only evaluate the “pessimistic” scenario where the post-quantum algorithms degrade the page performance, we did not consider TLS session resumption. We also evaluated only cached page measurements where the browser has cached all page resources it can and the content is loaded faster. These results were somewhat pessimistic as they would indicate higher performance degradation than for uncached pages. For completeness, we analyzed different scenarios based on RTT and network speed (i.e., 3G, 4G, Cable). “3G” was modeled as 1.6Mbps down, 768Kbps up, and 300ms RTT. “4G” was 5Mbps down, 1Mbps up, and 170ms RTT. “Cable” was 5 Mbps down, 1 Mbps up, and 28ms RTT. The page content was served over QUIC and TLS 1.3 connections, supported by the browser. We took into consideration each measured metric and the connections (delivering the content of the page) that would affect it. We assumed that all completed connections before a metric fires would affect the metric which is a worst-case scenario. Obviously, two parallel connections’ impact on a Web metric will not have an additive effect; we treated parallel connections as impacting a metric once. Sequential connections were considered to impact the metric cumulatively.

The metrics we focused on were TLS handshake, TTFB, LCP, and DC time. We extrapolated the post-quantum slowdown in each of these metrics by calculating the transmission delay of the additional data introduced in the handshake by post-quantum algorithms. We modeled this size as roughly 15KB. We assumed that the processing overhead of MLWE is immaterial to these connection as it is shown in various studies (Section II). We also analyzed the impact of reducing handshake size to an additional 10KB (“trimmed” certificates) by omitting intermediate CAs [18], [19].

Quantitative: Although the qualitative experiments gave us an intuition about the impact of MLWE on real webpages, they did not measure the actual page performance metrics. We wanted to build on the conclusions drawn from extrapolated data by experimenting with real webpages that support ML-KEM key exchanges and ML-DSA certificates.

As introducing ML-DSA authentication in real websites was premature for the WebPKI because the CA Browser Forum had not yet introduced it in their guidelines, we built our own experimental website. We chose to evaluate the pessimistic scenario with “lean” Web content that gets transferred over a few TLS connections. We built two variants of the page: a “slim” version that transferred a total of 91KB over six TLS connections and a “chubby” one of 221KB over five connections. The webpage was broken down into a few elements: main HTML/CSS content of 25 or 50KB, a basic JavaScript clock of 25 or 50KB, one 1KB favicon.ico image, a 3KB JavaScript that loaded the external content, three images of 5 or 25KB, and an image of 25 or 50KB. Figure 10 in

Appendix A shows a layout of the webpage in more detail. The size variations were created for the slim or chubby version of the web content. Two of the images were preloaded with the page (affecting the Fully Loaded metric). The other two images were lazy loaded with the page. Note that all four images were cacheable; a browser could cache them after its first visit and not have to fetch them with every visit. When caching was used the content was 100KB transferred over two connections for the chubby page and 51KB over one connection for the slim page. Depending on caching being used, the different connection counts also allowed us to experiment with the impact of each TLS handshake on the total page metric. That way, we created content that behave similarly to today’s webpages which are served over multiple parallel connections some of which are preloaded. Figure 8 in Appendix A shows `cnn.com` as an example whose content is fetched via multiple parallel connections.

Figure 9 in Appendix A shows the experimental setup in more detail. The Web content was served over five `nginx` servers with OpenSSL 3.5. The main content server was in the AWS `us-east1` region and the four auxiliary servers serving the images were in the AWS `us-east2` region. That way we ensured that there is some geographical distribution of the content as a more realistic condition. The Web client measurements were collected with `webpagetest.org` run from Virginia, US. The page content was served over TLS 1.3. For comparison, we tested classical and post-quantum handshakes. The classical handshakes had an ECDSA-based server cert chain issued by Let’s Encrypt for the server and used X25519 for key exchange. The quantum-resistant handshakes used X25519MLKEM768 key exchange and an 18KB bogus certificate chain along with the same ECDSA Let’s Encrypt certificate chain. In essence, we inflated the certificate chain to sizes equivalent to ML-DSA-65 WebPKI certificates and still used a classical ECDSA certificates for the server to be authenticated by the browser. The performance of ML-DSA signing and verification is very efficient and comparable to ECDSA, so this choice does not deem the testing inaccurate. To avoid unnecessary RTTs due to TCP congestion control, we increased the `cwnd` on the server which is commonly done by Web content providers today.

The metrics we focused on were TLS handshake time, TTFB, LCP, DC, Fully Loaded Time, and CLS. Each test was run 100 times except for the TLS handshake times which were collected 200 times. We extracted the median and the 90th percentile for each metric to deduce the impact on each.

B. mTLS Experiments

Our last experiment investigated the impact of post-quantum cryptography on mTLS applications, an area that has received limited attention in the literature. Unlike standard TLS, mTLS requires bidirectional certificate-based authentication, potentially doubling the cryptographic overhead. To evaluate the performance impact of post-quantum MLWE on mTLS, we deployed an experimental framework to emulate multiple real-world scenarios. Our setup and experimental results will be

made available on GitHub upon acceptance. To cover a broader range of scenarios, the parameters we tweaked for our experiments include network bandwidth, RTT, TCP *icwnd*, and the application data transferred. The network speeds include 1Gbps, 100Mbps and 1Mbps, representing high-performance and constrained environments. We considered relatively stable networks with jitter following a normal distribution with mean of $RTT/2$ and standard deviation of $RTT/90$. We assumed no packet loss. We considered RTTs of 35, 50, and 150ms representing localized and cross-regional connections. The *icwnd* was 10 or 24. In terms of application data, we analyzed the performance across four data transfer sizes from the server to the client - no application data transfer (0KB), essentially reporting TLS handshake times, 50KB, 150KB, and 400KB, which captures common applications that usually transfer a few hundreds KB. Our measured handshake times stopped at the server TCP ACK of the client *Finished* message which means three round-trips for a typical plain TLS 1.3 handshake.

In our mTLS experiments, we compared classical TLS ECDH P256 with RSA/ECDSA certificates against post-quantum X25519MLKEM768 with ML-DSA cert chains across all network configurations. We assumed leaf certificates and Signed Certificate Timestamps (SCT) based on ECDSA P-256 in the classical experiments and ML-DSA-44 in the post-quantum ones. Intermediate and root certificates were assumed to be RSA4096 and ML-DSA-65, respectively. The total certificate chain size for the post-quantum case amounted to about 22KB whereas the classical was about 3.5KB. To simulate the impact of two SCTs per server certificate, we instead expanded the certificate chain to two intermediate CA's both using ML-DSA-65.

For each experimental configuration, we collected two performance metrics: the TLS handshake time as the time to establish the mTLS session and the TTLB capturing the

total connection latency. We ran each experiment 1000 times and calculated the median and 90th percentile of the MLWE overhead for each metric to analyze the overall impact on mTLS applications.

V. PERFORMANCE RESULTS

Before presenting our experimental results, we give some intuition about large scale use cases which would not be significantly impacted by going over real-world AWS service examples where network bandwidth and latency are not a concern. Figure 3 shows actual aggregate connection data for three different AWS services in a certain US region. The graphs report the 50th (P50) and 90th (P90) percentiles of the downloaded and uploaded data sizes per TLS connection terminated on the relevant AWS service endpoints. The bytes reported do not include the TLS handshake.

Figure 3a shows that TLS connections to Service 1 endpoints download from 1KB (P50) to 75KB (P90) and upload from 5KB (P50) to 275KB (P90). The additional data in post-quantum TLS handshakes would impact mostly the lower percentiles as they exceed the actual application data transferred. The 50th percentile of the handshake times was about 5ms and 12ms at the 90th which shows that these connections happen over high-speed connections with jumbo frames and small round-trips in the data center. Thus, although the additional PQ data would “waste” bandwidth, when compared with the application data, it would still be transferred with transmission delay ranges of just a few milliseconds. These connections stayed up for 7.5 and 12.5s at the 50th and 90th percentiles, respectively. The long duration is because the application performs other operations after receiving the data or sometimes delays closing the connection which strengthens the argument that millisecond-level delays for the post-quantum handshakes

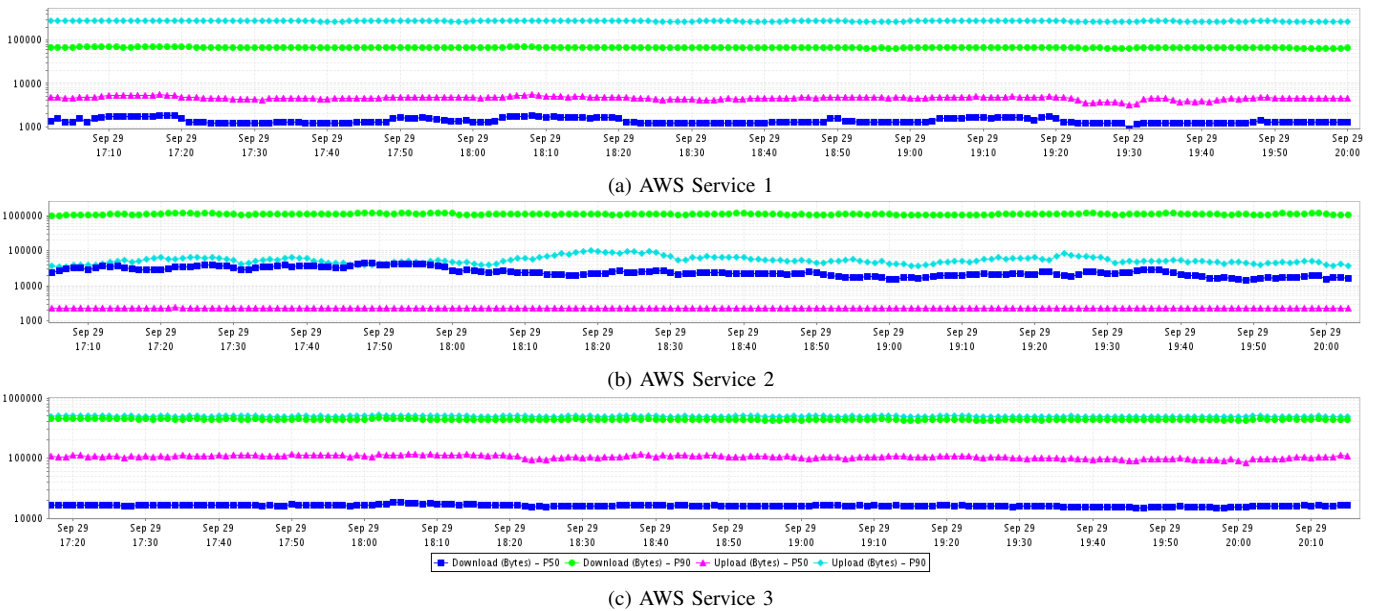


Fig. 3: Aggregated TLS connection download/upload data size percentiles in bytes for three AWS services in one US region.

will not make a difference. The median of the requests per connection was 1 and the 90th percentile was 100.

Similar conclusions can be drawn for Services 2 and 3. Service 2 endpoints saw download data sizes ranging from 15KB (P50) to 1MB (P90) and upload from 2KB (P50) to 100KB (P90) as seen in Figure 3b. The handshake time was 5 and 22ms at the 50th and 90th percentiles, respectively. The requests per connection were 1-2 at the 50th percentile and 3-4 at the 90th. The duration of the connections had a median of 300ms and 7.5s at the 90th percentile. Service 3, shown in Figure 3c, had downloads of 10KB (P50) and 400KB (P90), while upload transfers spanned from 50KB (P50) to 500KB (P90). The handshake time was 3 and 10ms at the 50th and 90th percentile, respectively. Each connection was seeing 75 requests per connection at the 50th percentile and 80 at the 90th. These connections lifetimes had a 50th percentile of 1s and 90th of 1min. These examples show that the additional data introduced by MLWE would introduce delays ranging in the milliseconds which are unlikely to be noticed by applications using multiple round-trips and transferring large amounts of data under fast network conditions.

A. Web Metrics

Qualitative: Table I shows the results of the post-quantum Web metric degradation extrapolated from actual data collected for popular websites. We see that ML-DSA certificate chains of size (~ 22 KB), measured over good speed, high RTT networks (4G) would introduce a 7.8% slowdown in TLS handshakes. The impact would drop to 5.2% with trimmed ML-DSA certificate chains. That becomes $\sim 57\%$ for Cable networks with small RTTs simply because the transmission delay of the additional data is more pronounced with small round-trip times. For high RTT, slower networks (3G), it would become $\sim 17\%$ as the transmission delay increases with the slower speeds but also gets absorbed by the RTT. Note that the TLS handshake time is an indicator of speed for the page, but it is not a metric that is directly tied to user experience as the user has not seen any of the downloaded content at that time. Similarly, the TTFB includes the duration of fetching the first element of the page, and we can see that the additional time would amortize the TLS handshake overhead for all pages to under 3% in the 4G experiments. That is mostly because it includes one or more additional round-trips.

Looking at the page sizes, we notice that in their cached versions, most pages download sizable total amounts of data which reinforces the intuition that 15 additional KB for the post-quantum handshakes will not affect the user experience too much. Some of these pages downloaded smaller content under 50KB. These would be more affected by the MLWE algorithms, but we should note they were all cached versions of the landing page of the websites. The logged-in versions were much larger.

When looking at the LCP metrics, we notice that they would drop below the TTFB which is because the LCP involves downloading and rendering more content and more round-trips which amortize the cost of the post-quantum handshakes.

LCP is the time the user perceives that the main content is loaded. Thus, it is obvious that the user will notice the post-quantum slowdown much less than the TLS handshake slowdown could lead us to believe. Our extrapolated FCP metric impact was similar to the LCP. The DC time would stay in the same range as the LCP mostly because these landing pages did not include a lot of content and the Load time was coinciding with the LCP or included just a little more content. We note that the degradation would be more prevalent in the Cable scenario because the post-quantum data transmission delay is in the same range as the RTT. Some of the slowdowns that exceed 3% were either because the content affecting the metric was insignificant in size and/or there were sequential connections delivering the content which made the PQ slowdown cumulative.

In summary, the sizable amounts of Web content included in each page and multiple round-trips and browser rendering times affecting most Web metrics are expected to mask the impact of post-quantum algorithms on the TLS handshakes. Networks with slower speeds and smaller round-trips will be impacted more. Webpages served over small sequential connections will also notice more degradation by the new algorithms.

It is worth noting that, although not investigated in this analysis, a portion of Web connections use TLS session resumption which does not require server certificate authentication and thus will see minimal impact. [36] reported 37% of connections use resumption. Firefox telemetry data showed 20-25%. AWS CloudFront saw similar ranges. Thus, over a quarter of Web connections will not see any impact from ML-DSA.

[36] also reported a small median for the data transfer size (<10 KB) of Web connections. This would mean that more than half of the Web connections would be “wasting” more data setting up the post-quantum handshake than transferring content, and, thus, would be impacted significantly. The reported median was an aggregate statistic that did not give an indication of the connection size per webpage. This contradicts the sizes observed in Table I for the sample pages. On the other hand, httparchive.org reported a median size of $\sim 2.5\text{MB}/12 \sim 200\text{KB}$ per page which has been growing over the years which also contradicts the reported median in [36]. To investigate it further, we looked into the “logged in” versions of 12 popular websites like amazon.com, gmail.com, youtube.com, instagram.com, x.com, github.com, and reuters.com. Looking into the TLS or QUIC flow sizes, we observed that they still include a few connections transferring large sums of Web content, and many smaller connections transferring just a few KB of data. These “lean” connections included analytics collection, ads, user or geolocation tracking, HTTP errors, HTTP 304 for cached content or HTTP redirects, small JavaScript, JSON, or images. This empirical analysis was not a broad Internet study on a per-page basis, but it could explain the median connection sizes reported [36] and the per page content reported in httparchive.org. If this explanation holds for the major-

Website	4G				4G (trimmed)			Cable (trimmed)			3G (trimmed)		DC [KB]
	TLS (+%)	TTFB (+%)	LCP (+%)	DC (+%)	TLS (+%)	LCP (+%)	DC (+%)	TLS (+%)	LCP (+%)	DC (+%)	TLS (+%)	DC (+%)	
google	7.8	1.72	1.11	0.55	5.2	0.74	0.37	57.1	2.92	1.78	16.67	2.33	445
youtube	7.8	1.76	1.11	1.27	5.2	0.74	0.84	57.1	2.11	2.11	16.67	2.88	36
outlook	7.8	1.08	0.98	1.02	5.2	0.66	0.68	57.1	2.76	3.34	16.67	1.70	170
facebook	7.8	1.30	1.14	0.84	5.2	0.76	0.56	57.1	2.44	2.11	16.67	1.98	30
docs.google	7.8	0.90	0.67	0.91	5.2	0.45	0.60	57.1	1.86	2.83	16.67	1.68	620
linkedin	7.8	1.13	0.91	0.84	5.2	0.61	0.56	57.1	1.82	1.68	16.67	2.31	15
chatgpt	7.8	1.31	0.49	0.69	5.2	0.33	0.46	57.1	1.28	1.23	16.67	2.01	18
instagram	7.8	1.56	0.57	0.70	5.2	0.38	0.47	57.1	1.09	1.15	16.67	1.96	96
x	7.8	1.76	1.70	1.74	5.2	1.14	1.16	57.1	3.87	3.74	16.67	2.95	144
wikipedia	7.8	0.91	1.13	1.09	5.2	0.75	0.72	57.1	3.19	2.96	16.67	2.31	583
cnn	7.8	2.29	0.96	1.04	5.2	0.64	0.70	57.1	1.50	1.95	16.67	3.92	2,300
msn	7.8	1.40	1.72	1.05	5.2	1.15	0.70	57.1	2.18	2.37	16.67	1.29	590
stackoverflow	7.8	1.78	0.92	1.33	5.2	0.61	0.89	57.1	2.34	2.97	16.67	1.93	33
tiktok	7.8	1.53	0.43	0.81	5.2	0.29	0.54	57.1	1.77	2.34	16.67	2.81	72

TABLE I: Extrapolated Web Metric degradation due to ML-DSA (+15KB), “Trimmed” ML-DSA (10KB) authentication over classical RSA in 4G (9Mbps/9Mbps, 170ms RTT), Cable (5Mbps/1Mbps, 28ms RTT), 3G (1.6Mbps/768Kbps, 300ms RTT) networks.

ity of the Internet, it would mean that webpages indeed transfer large content sizes which intuitively will amortize the cost of the post-quantum algorithm sizes. The “lean” connections delivering content “peripheral” to the Web will notice more degradation. It is unlikely that the delivery of “peripheral” content like analytics, tracking, and ads will really impact user experience metrics, but more experimentation is necessary to confirm this intuition.

Quantitative: After the qualitative analysis regarding MLWE’s user experience degradation in popular websites, we present the results with the experimental webpages discussed in Section IV-A. Table II reports the comparison between the measured median and 90th percentiles of the Web metrics of pages delivered over classical and PQ TLS 1.3 connections. The metrics were measure for the different variants of the experimental webpage (i.e., slim/chubby, uncached/cached) under different network conditions. We can see that the impact on the TLS handshake is 8-10% for the 4G (i.e., high RTT,

good speed) and 47-49% for the Cable (i.e small RTT, slower speed) scenarios. The slowdown is high for Cable because the transmission delay of the additional PQ data in the handshake is in the same range as the round-trip. Note that the slowdown at the 90th percentile for Cable is much higher than 4G. The reason for the discrepancy is the number of experiments. The `webpagetest.org` quota limited the tests we could run. At 200 experiments, the higher percentiles deteriorated earlier for the Cable TLS handshakes and looked far more impacted. In more detail, the 82nd percentile was 58% slower but it went back to 30% at the 95th. Repeating these tests thousands of times would have led to more aligned distributions between 4G and Cable.

The TTFB is far less impacted than the handshake. It remains below 3% for the 4G connections and under 15% for Cable. The small RTT is the reason for the higher impact on the Cable experiments. The LCP drops even further. Be reminded that the LCP time is when the user conceives the main page to have been loaded. At its more pronounced overhead of 8%, we notice that the LCP slowdown of the uncached webpage in the 4G network corresponds to 29ms. It is unclear that is 29ms will be noticeable in a metric that amounted to 363ms for the traditional connections. Note that LCP values under 2,500ms are considered “GOOD”. It is worth pointing out that the 90th TTFB and LCP percentiles were faster for the post-quantum experiments. This was due to our testing. At 100 experiments, the classical connections deteriorated faster. Looking at the rest of the percentiles, the classical TTFB and LCP times were increasing faster for the classical connections (>70th percentile) and were again dropping after the 90th. If we had run the tests thousands of times, we expect the PQ measured percentiles to be higher than the classical ones.

The Load and Fully Loaded times showed similar patterns. The post-quantum impact is lower than the handshake itself. They are slightly higher than the LCP because they are impacted by more TLS connections transferring Web content,

	Handshake		TTFB		LCP		Load time		Fully Loaded Time		CLS
	50 th	90 th	50 th	90 th	50 th	90 th	50 th	90 th	50 th	90 th	90 th
4G	51 KB 1 conn†		+2.3%	+2.6%	+2% (+18ms)	+0.6% (+4ms)	+1.6%	+0.9%	+1.6%	+0.9%	0%
	91 KB 5 conns‡	+8.8% +12%	+1.7%	-5.7%	+0.6%	-2.9%	+1.4%	+3.7% (+76ms)	+2.8% (+75ms)	+4.5% (+128ms)	0%
	100 KB 2 conns†		+2.1%	+1.9%	+2.3% (+21ms)	+1.5% (+14ms)	+1.7%	+1.4%	+1.7%	+1.4%	0%
	221 KB 6 conns‡ 2 conns*	+8.8% +10%	+2.1%	-13%	+0.4%	-4.7%	+0.6%	-2.6%	+0.4%	-1.6%	0%
Cable	51 KB 1 conn†		+13%	+5.9%	+2.4% (+1ms)	+0.9% (+4ms)	+6.3% (+26ms)	+6.3% (+29ms)	+6.3% (+26ms)	+6.3% (+29ms)	114%
	91 KB 5 conns‡	+47% +42%	+11%	+9%	+5.7% (+21ms)	+0.9% (+4ms)	+11% (+69ms)	+8.2% (+58ms)	+12% (+103ms)	+9.6% (+90ms)	0%
	100 KB 2 conns†		+14%	+8.1%	+0.3% (+1ms)	+2.2% (+9ms)	+5.4% (+27ms)	+6.8% (+37ms)	+5.4% (+27ms)	+6.8% (+37ms)	0%
	221 KB 6 conns‡ 2 conns*	+49% +76%	+16%	+4.5%	+8% (+29ms)	+2.7% (+12ms)	+7.0% (+54ms)	+6.0% (+50ms)	+8.6% (+88ms)	+5.5% (+61ms)	0%

TABLE II: Experimental webpage Web Metric degradation due to MLWE key exchange and authentication in 4G (9Mbps/9Mbps, 170ms RTT) and Cable (5Mbps/1Mbps, 28ms RTT) networks. † denotes cached connections; ‡ denotes lazy loaded connections; * denotes parallel connections

so the post-quantum data has introduced more slowdown, although these connections were parallel. Even the higher impact at 3.7% for the 90th percentile of the slim uncached page of the 4G scenario and the 50th of the slim uncached page for the Cable scenario amount to 76ms and 69ms respectively. 79ms in addition to 1,298ms and 69ms to 634ms are not likely to be noticeable by human users. Similarly for the Fully Loaded times, the slowest 4.9% and 12% are an additional 128ms on 2,811ms and 103ms on 944ms. These absolute time slowdowns are probably not going to be noticeable. Note that the Fully Loaded time is a metric that does not tie to user experience, but is an indicator of how fast the total content is loaded.

The CLS impact remained 0% except for the 90th percentile of the slim cached page for the Cable scenario. The reason for that was a race condition between rendering the two cached images and the fetching and rendering of the HTML/CSS static content over the post-quantum connection which led to the shifting of the page for some of the MLWE experiments.

In summary, our experiments with a real webpage under different network conditions showed that the MLWE algorithms will affect the Web metrics much less than the TLS handshake because these metrics include other factors like downloading the content, DNS lookups, request-response time, and rendering the content. Faster networks and longer RTTs will amortize the impact more. This does not mean the Web user metrics will not be affected and that trimming down the post-quantum authentication data in the handshake should be avoided, but overall the user will not notice tens of milliseconds of slowdowns of the metrics which amount to hundreds of milliseconds already.

B. mTLS

Now we discuss the results of the mTLS experiments that tested different network conditions and application data sizes. Note that although many mTLS use cases employ public WebPKI server certificates which include SCTs and result in certificate chains around 22KB, most client certificate chains consist of private PKI certificates which do not include SCTs. This results in slightly smaller client certificate chain sizes. In this work, we used ~22KB ML-DSA certificate chains on the server side and ~14.5KB on the client side to report the handshake and TTLB impact.

Figure 4 shows the PQ migration impact on the median of plain TLS compared to the median of mTLS handshakes in fast and slow networks. The server's initial congestion window was set to 24MSS, while the client's defaulted to 10MSS. In fast network conditions (100Mbps), the PQ TLS handshake was under 2ms slower (+1.23%) than the plain classical TLS handshake. For mTLS, the PQ handshake was approximately 51ms (33.05%) slower than the classical due to the additional round-trip needed to transmit the client's PQ certificate chain exceeding the *icwnd*. We mark each round-trip with a horizontal dashed line, where the red lines denote additional RTTs required for large certificate transmission compared to classical handshakes. We notice that the small

icwnd was the bottleneck causing the slowdown. In slow network conditions (1Mbps), the performance drop is even more significant, ranging between 82.65% and 115.29% for plain TLS and mTLS, respectively, because the slowdown consists of the longer transmission time for the PQ data chain in each direction. Another interesting finding is that the additional RTT does not appear in the slow network speed scenario because the transmission delay slows down the transfer and prevents the client from filling its *icwnd*. We discuss this further in Figure 11 in Appendix A.

Figure 5 reports the 90th percentile of handshake (0KB) and TTLB (50KB, 150KB, 400KB) times for mTLS 1.3 connections with 50ms RTT under different network speeds with Server and Client *icwnd* set to 24MSS and 10MSS, respectively. In the fast network, we can see that the additional round-trip adds noticeable slowdowns to the handshake (33%, also in Figure 4). As the data transferred increases, the impact drops and amounts to about 10% when the application downloads 400KB because the application data transmission amortizes the cost of the round-trip. Be reminded that the *icwnd* is often configured high on the server side to optimize content delivery performance based on network conditions [40], [41],

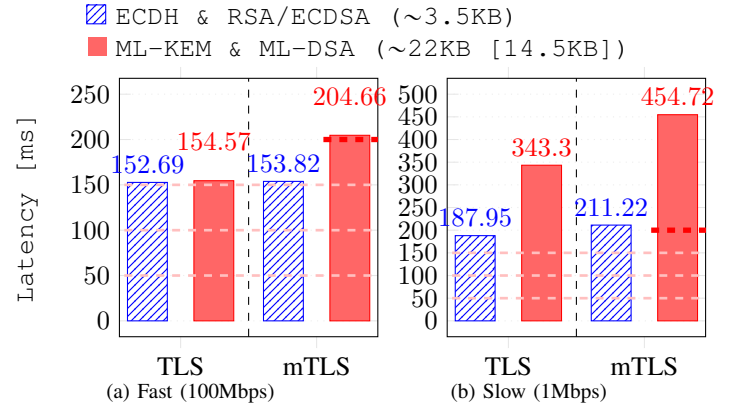


Fig. 4: Median for plain TLS vs. mTLS Handshake Times, Client *icwnd*=10, Server *icwnd*=24, RTT=50ms.

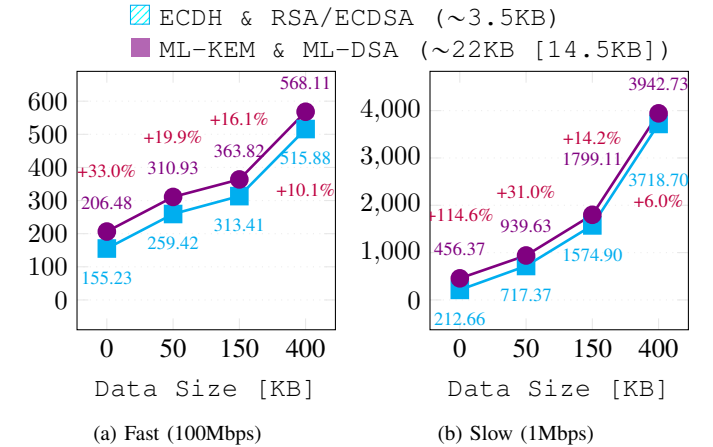


Fig. 5: 90th percentile of mTLS 1.3 TTLB latency in ms for varying application data sizes and network speeds. RTT=50ms. Server *icwnd*=24, Client *icwnd*=10.

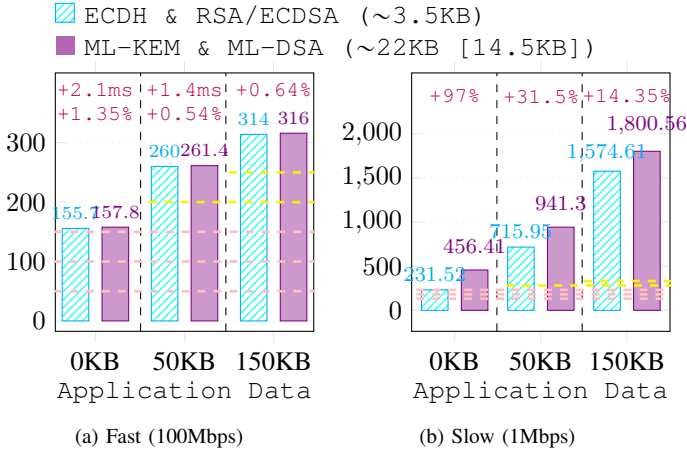


Fig. 6: 90th percentile of mTLS 1.3 TTLB latency in ms for varying application data sizes and network speeds. RTT=50ms. Client/Server *icwnd*=24.

[42, §Appendix C], but mTLS clients may typically retain the default *icwnd*=10MSS [37]. They may need to increase it when migrating to ML-DSA X.509 certificate chains to prevent the additional RTT. In the slow network, mTLS suffers much more (114%) at the handshake time because of the slower transmission time of the post-quantum certificate chains. The overhead drops as the application data increases similarly to the fast connection.

Figure 6 shows the same metrics when both the client and the server use a large *icwnd*. At high network speeds, the MLWE slowdown is minimal because it only includes the additional transmission delay of the PQ data without an additional round-trip. When the network speed is slow, the impact is much more pronounced and gets amortized as the application data grows.

Figure 7 shows the TTLB for 50KB of application data in fast and slow networks and for different RTTs. In fast networks and small RTT=35ms, the PQ overhead is under 1% since the transmission time for the additional MLWE data in the

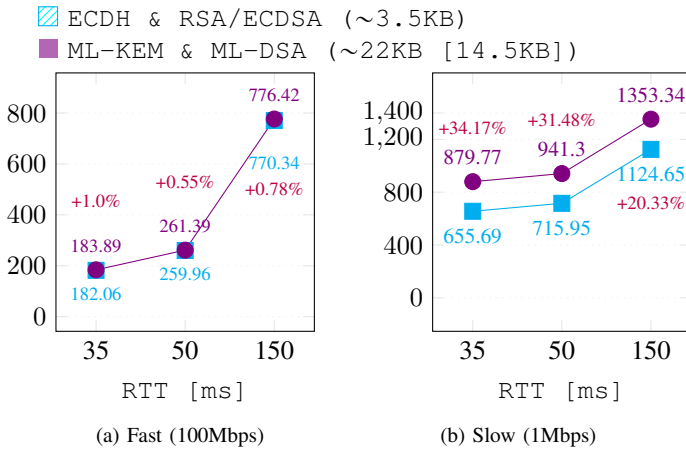


Fig. 7: 90th percentile of mTLS 1.3 TTLB latency in ms for 50KB of application data over different RTT values. Client/Server *icwnd*=24.

handshake is small. The impact drops as the RTT increases because the data transmission cost become insignificant. In slow networks, the constant ~225ms transmission overhead of the MLWE data translates to ~35% slowdown for RTT=35ms and drops to ~20% when RTT=150ms. It is obvious that as RTTs increase and networks get faster, the PQ data cost become less important.

In summary, the mTLS experiments demonstrated that as application data amortizes the post-quantum, mTLS connections experience greater impact than plain TLS since they transfer around double the post-quantum keys and signatures. Network bandwidth significantly affects performance; slow upload or download speeds cause higher slowdowns due to transmission delays. When application data is much larger than the PQ handshake data, the overhead becomes negligible. On the other hand, slim connections transmitting small amounts of data suffer more, especially under slower network conditions. Optimizations such as session resumption and CA certificate suppression could mitigate these effects. Our study showed similar results for median and 90th percentiles due to low network volatility in our experiments. More unstable networks would demonstrate worse impact at higher percentiles, but network instability was shown to affects both classical and post-quantum TTLBs [35].

VI. CONCLUSION

The transition to post-quantum MLWE key exchange and authentication in applications running over TLS-based protocols will somewhat affect application performance. In this work, we argue that this impact will remain unnoticed by the end user in many cases. We evaluated three AWS services that run under high-speed, small RTT conditions where the PQ impact will be insignificant. We performed qualitative and quantitative analysis for Web content and showed that it often involves different connections some of which fetch large content and others transfer small amounts of data for analytics, ads, tracking, redirects, and more. Although MLWE will slow down the latter lean connections, that will not be directly perceived by the end user. Our evaluation of user-centric Web metrics revealed that the slowdown for the Web will remain small since the PQ delays are amortized by all the other delays affecting Web metrics.

Finally, we performed mTLS 1.3 experiments where we showcased that although applications will see almost double the handshake slowdown compared to plain TLS 1.3 handshakes, the overhead will diminish as applications transfer more data, especially at fast network speeds and large *icwnd*. Naturally, the impact will be more noticeable at lower upload or download speeds and smaller RTTs. We also showed that proper *icwnd* settings can prevent additional RTTs.

REFERENCES

- [1] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- [2] NIST. NIST PQ project. <https://csrc.nist.gov/projects/post-quantum-cryptography>, August 2016.

- [3] National Institute of Standards and Technology. Fips 203: Module-lattice-based key-encapsulation mechanism standard, August 2024. Accessed: 2025-05-19.
- [4] National Institute of Standards and Technology. Fips 204: Module-lattice-based digital signature standard, August 2024. Accessed: 2025-05-19.
- [5] Panos Kampanakis and Dimitrios Sikeridis. Two post-quantum signature use-cases: Non-issues, challenges and potential solutions. *Cryptology ePrint Archive*, Report 2019/1276, 2019.
- [6] Google. Understanding Core Web Vitals and Google search results. <https://developers.google.com/search/docs/appearance/core-web-vitals>.
- [7] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. Zero Trust Architecture. Technical Report NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
- [8] Adam Langley. CECPQ1 results, November 2016.
- [9] Adam Langley. CECPQ2, December 2018.
- [10] Luke Valenta Kris Kwiatkowski. The TLS Post-Quantum Experiment, October 2020.
- [11] Eric Crockett, Christian Paquin, and Douglas Stebila. Prototyping post-quantum and hybrid key exchange and authentication in tls and ssh. In *NIST Second PQC Standardization Conference*, 2019.
- [12] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in tls. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography*, pages 72–91, Cham, 2020. Springer International Publishing.
- [13] Nina Bindel, Udyani Herath, Matthew McKague, and Douglas Stebila. Transitioning to a quantum-resistant public key infrastructure. In *Post-Quantum Cryptography: 8th International Workshop, PQCrypto 2017*, pages 384–405. Springer, 2017.
- [14] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum Authentication in TLS 1.3: A performance study. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020.
- [15] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in tls 1.3. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security, ASIA CCS '22*, page 727–740, New York, NY, USA, 2022. Association for Computing Machinery.
- [16] Panos Kampanakis and Michael Kallitsis. Faster post-quantum TLS handshakes without intermediate CA certificates. In Shlomi Dolev, Jonathan Katz, and Amnon Meisels, editors, *Cyber Security, Cryptology, and Machine Learning*, pages 337–355, Cham, 2022. Springer International Publishing.
- [17] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum tls: An approximate membership querying approach. In *Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies, CoNEXT '22*, page 35–42, New York, NY, USA, 2022. Association for Computing Machinery.
- [18] Dennis Jackson. Abridged Compression for WebPKI Certificates. Internet-Draft draft-ietf-tls-cert-abridge-02, Internet Engineering Task Force, September 2024. Work in Progress.
- [19] Bob Beck, David Benjamin, Devon O'Brien, and Kyle Nekritz. TLS Trust Anchor Identifiers. Internet-Draft draft-ietf-tls-trust-anchor-ids-02, Internet Engineering Task Force, September 2025. Work in Progress.
- [20] David Benjamin, Devon O'Brien, Bas Westerbaan, Luke Valenta, and Filippo Valsorda. Merkle Tree Certificates. Internet-Draft draft-davidben-tls-merkle-tree-certs-08, Internet Engineering Task Force, October 2025. Work in Progress.
- [21] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In *Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies, CoNEXT '20*, page 149–156, New York, NY, USA, 2020. Association for Computing Machinery.
- [22] Cloudflare Bas Westerbaan. Sizing Up Post-Quantum Signatures, November 2021.
- [23] David Adrian. Post-quantum cryptography is too damn big. <https://davidian.io/blog/posts/pqc-signatures-2024/>, March 2024.
- [24] Tzinos Iraklis, Konstantinos Limniotis, and Nicholas Kolokotronis. Evaluating the performance of post-quantum secure algorithms in the TLS protocol. *Journal of Surveillance, Security and Safety*, 3(3):101–127, 2022.
- [25] Ronny Döring and Marc Geitz. Post-quantum cryptography in use: Empirical analysis of the tls handshake performance. In *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pages 1–5, 2022.
- [26] Markus Sosnowski, Florian Wiedner, Eric Hauser, Lion Steger, Dimitrios Schoinianakis, Sebastian Gallenmüller, and Georg Carle. The performance of post-quantum tls 1.3. In *Companion of the 19th International Conference on Emerging Networking EXperiments and Technologies, CoNEXT 2023*, page 19–27, New York, NY, USA, 2023. Association for Computing Machinery.
- [27] NIST NCCOE PQC Migration Project. NIST SPECIAL PUBLICATION 1800-38C Draft.
- [28] José A Montenegro, Ruben Rios, and Javier López-Cerezo. A performance evaluation framework for post-quantum tls. *Future Generation Computer Systems*, page 108062, 2025.
- [29] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum tls on embedded systems: Integrating and evaluating kyber and sphincs+ with mbed tls. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, ASIA CCS '20*, page 841–852, New York, NY, USA, 2020. Association for Computing Machinery.
- [30] Dominik Marchsreiter and Johanna Sepúlveda. Hybrid post-quantum enhanced tls 1.3 on embedded devices. In *2022 25th Euromicro Conference on Digital System Design (DSD)*, pages 905–912, 2022.
- [31] George Tasopoulos, Jinhui Li, Apostolos P Fournaris, Raymond K Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum tls 1.3 on resource-constrained embedded systems. In *International Conference on Information Security Practice and Experience*, pages 432–451. Springer, 2022.
- [32] George Tasopoulos, Charis Dimopoulos, Apostolos P Fournaris, Raymond K Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum tls 1.3 for resource-constrained embedded devices. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*, pages 366–374, 2023.
- [33] Jessica Bozhko, Yacoub Hanna, Ricardo Harrilal-Parchment, Samet Tonyali, and Kemal Akkaya. Performance evaluation of quantum-resistant tls for consumer iot devices. In *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, pages 230–235. IEEE, 2023.
- [34] Panos Kampanakis and Tancrede Lepoint. Vision paper: Do we need to change some things? In Felix Günther and Julia Hesse, editors, *Security Standardisation Research*, pages 78–102, Cham, 2023. Springer Nature Switzerland.
- [35] Panos Kampanakis and Will Childs-Klein. The impact of data-heavy, post-quantum tls 1.3 on the time-to-last-byte of real-world connections. Technical report, MADweb 2024, 2024.
- [36] Cloudflare Bas Westerbaan, Luke Valenta. A look at the latest post-quantum signature standardization candidates, November 2021.
- [37] Jerry Chu, Nandita Dukkkipati, Yuchung Cheng, and Matt Mathis. Increasing TCP's Initial Window. RFC 6928, April 2013.
- [38] Eric Rescorla, Hannes Tschofenig, and Nagendra Modadugu. The Datagram Transport Layer Security (DTLS) Protocol Version 1.3. RFC 9147, April 2022.
- [39] Martin Thomson and Sean Turner. Using TLS to Secure QUIC. RFC 9001, May 2021.
- [40] Mark Allman. Removing TCP's Initial Congestion Window. Internet-Draft draft-allman-tcpm-no-initwin-00, Internet Engineering Task Force, November 2015. Work in Progress.
- [41] Nicolas Kuhn, Emile Stephan, Gorrry Fairhurst, Raffaello Secchi, and Christian Huitema. Convergence of Congestion Control from Retained State. Internet-Draft draft-ietf-tsvwg-careful-resume-24, Internet Engineering Task Force, October 2025. Work in Progress.
- [42] Dr. Joseph D. Touch, Michael Welzl, and Safiqul Islam. TCP Control Block Interdependence. RFC 9040, July 2021.
- [43] Google Chrome. pq-tls-benchmark. <https://developer.chrome.com/docs/crux/methodology/metrics>.

APPENDIX

We now present some more details about the Web metric qualitative and quantitative experiments. Figure 8 shows a snippet from webpagetest.org of the connections

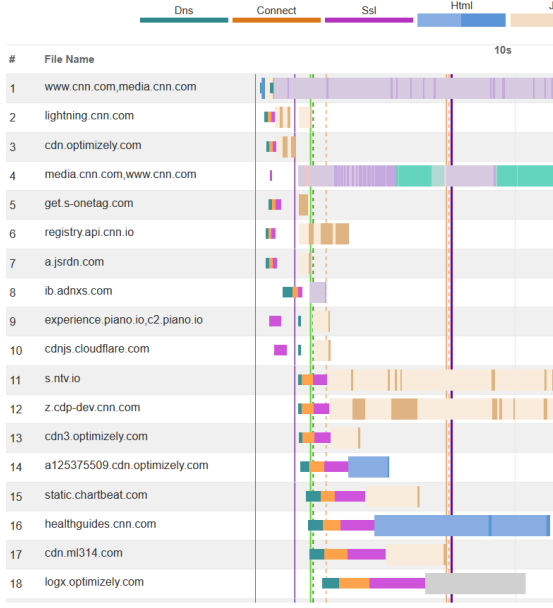


Fig. 8: Subset of parallel connections fetching `cnn.com`.

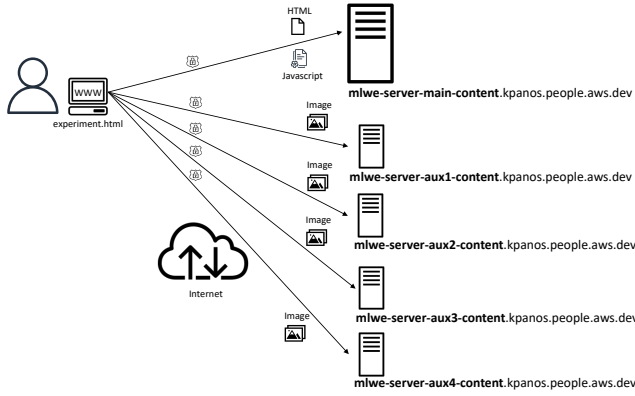


Fig. 9: Experimental webpage setup with based on `nginx` and `OpenSSL 3.5` on the server side and `webpagetest.org` as the client.

opened by a Desktop browser when visiting `cnn.com`. We can see a set of parallel connections. Such connections opened independently of each other will not have cumulative impact on web metrics. Intuitively, a webpage loading images from different destinations, uploading analytics data or ads asynchronously will not have its LCP or DC times affected by all of these connections.

Figure 9 represents how the experimental web content was served for the quantitative web performance tests (Section IV-A). It shows the main server serving the web content from the AWS `us-east1` region and the servers serving the four images from the AWS `us-east2` region.

Figure 10 demonstrates the experimental pages used in our quantitative web experiments (Section IV-A). It graphically shows the two JavaScript files, the little `favicon.ico` image, the HTML/CSS content and the images fetched from the four

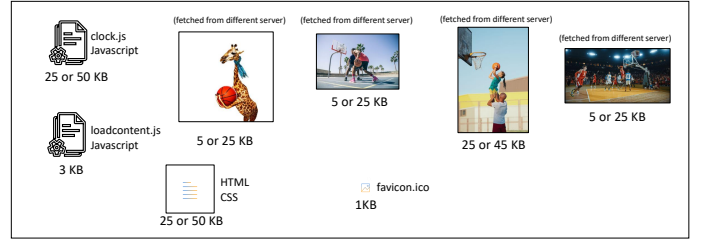


Fig. 10: Experimental webpage.

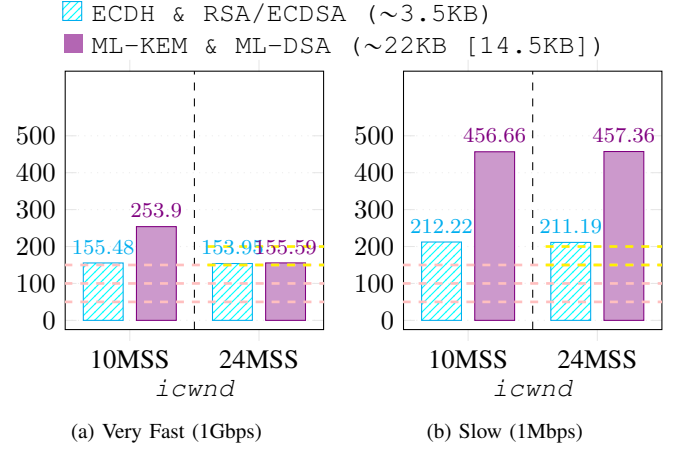


Fig. 11: 90th percentile of mTLS 1.3 handshake time for varying initial congestion windows and network speeds. `RTT=50ms`.

auxiliary servers and their sizes.

Finally, Figure 11 demonstrates the classical and PQ mTLS 1.3 handshake time for different `icwnd` and network speed and reveals an interesting interaction between the two. We noticed this interaction in Figure 4 too. In the 1Gbps network speed scenario, when the `icwnd` is small, the PQ mTLS handshakes include two additional round-trips ($2 \times 50\text{ms}$) due to the size of the PQ key exchange and authentication data sizes from the client and the server. In the slow network case, the PQ handshakes include the transmission time of the PQ handshake data, but not the two full additional RTTs. The total transmission time of $\sim 14.5\text{KB}$ cert chain from the client and $\sim 22\text{KB}$ from the server at 1Mbps is $\sim 250\text{ms}$ which is approximately the difference between the classical and PQ mTLS handshake time when there `icwnd=24`. For `icwnd=10`, we would expect the PQ handshake to be $250 + 2 \times 50\text{ms}$ slower due to the additional RTTs, but it remains about 250ms. The reason is that at slow speeds, the network slows down the transfer enough so that ACK packets arrive from the peer without the sender filling up the window. Intuitively, the slowness that would be introduced by mechanisms to prevent congestion is masked by the slow transmission itself.